

Grafos: Búsquedas (Semana 12)

Brenner Ojeda
bojeda@utec.edu.pe

Búsqueda en grafos

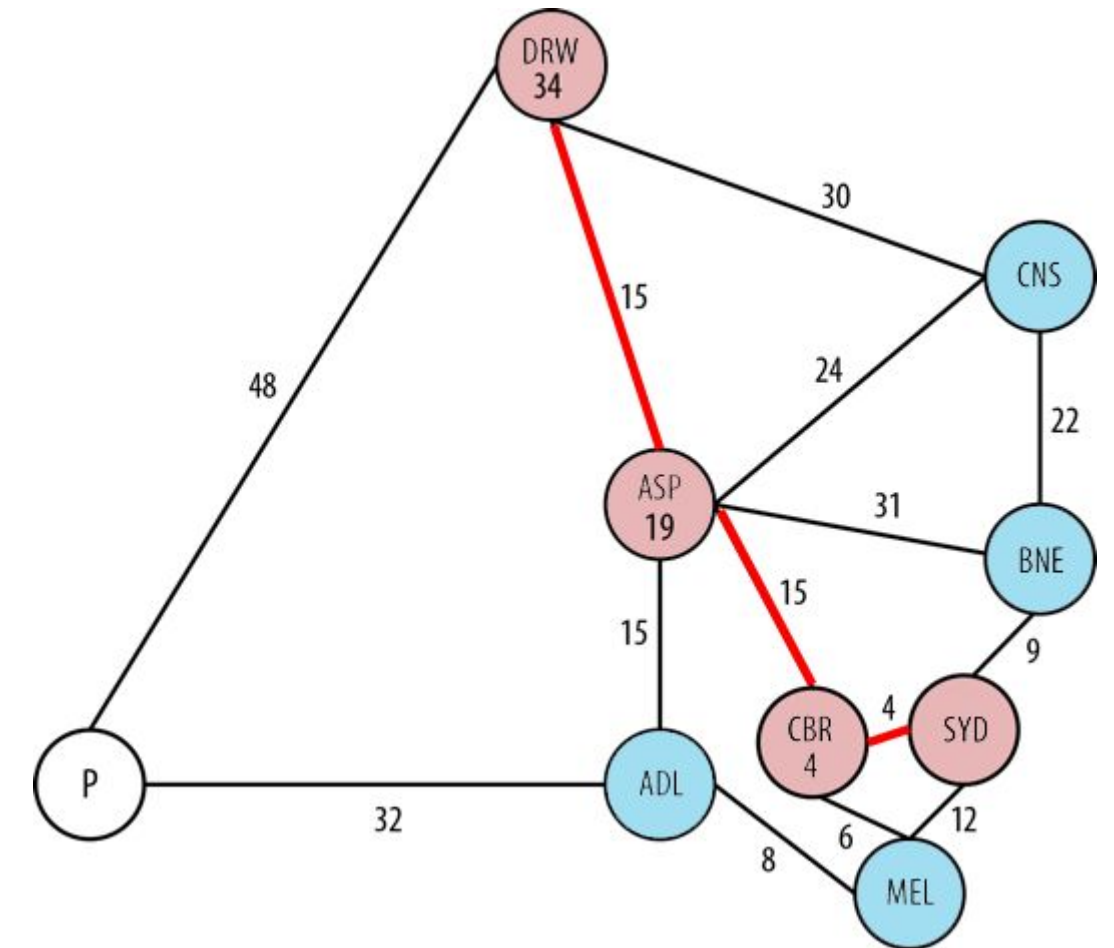
Se refiere a la exploración en un grafo.

Cada manera de explorar un grafo puede utilizar una estructura de apoyo diferente

El peor de los casos es $O(|V|^2)$

Para cada caso se utilizará memoria como ayuda. Recordando los vértices que ya han sido visitados

Se tiene varias aplicaciones: encontrar los vértices conexos, encontrar ciclos, caminos más cortos, etc

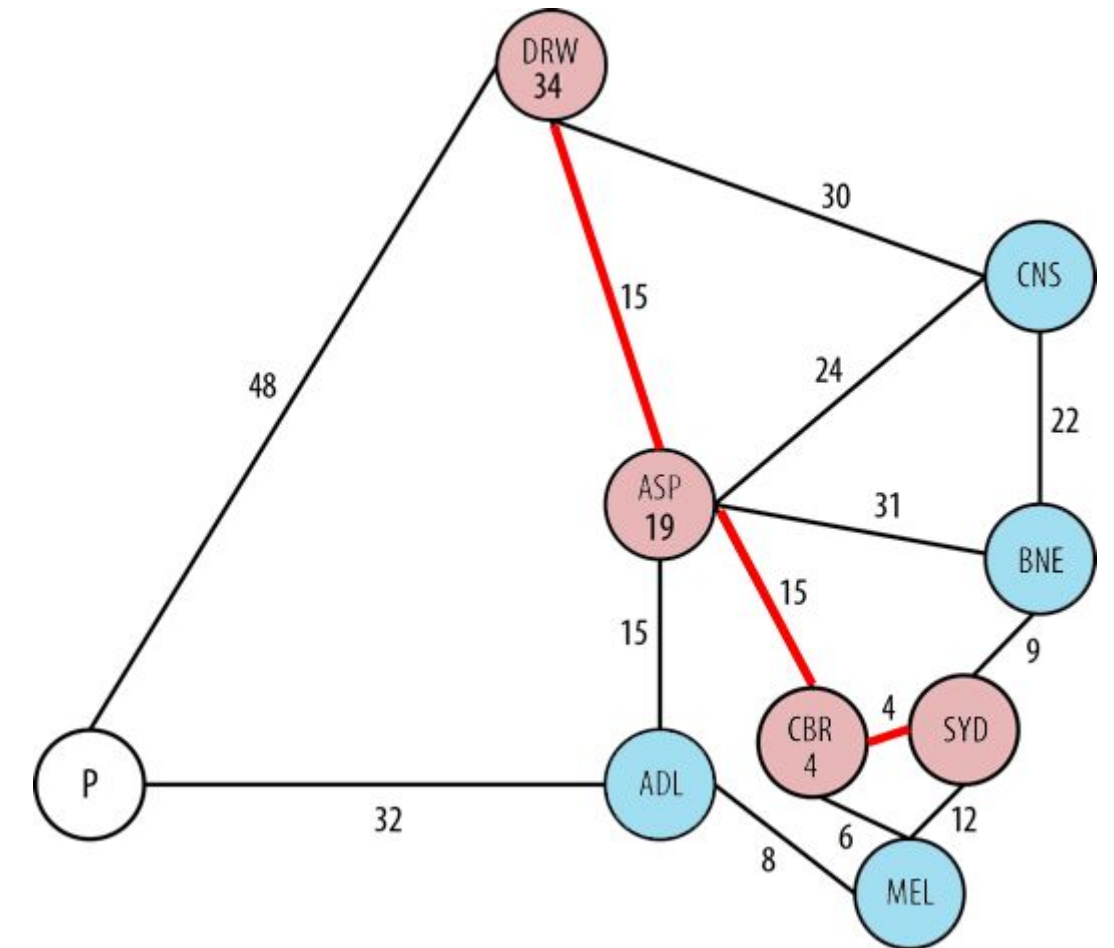


Búsqueda en grafos

La idea básica es ir expandiendo un árbol de estados al generar sucesores (estados) de un vértice ya visitado.

Cada estado es evaluado para saber si ya cumplimos con nuestro objetivo.

Se debe usar alguna heurística para generar sucesores, como el siguiente vértice al que se llega con menor peso, o uno aleatorio.

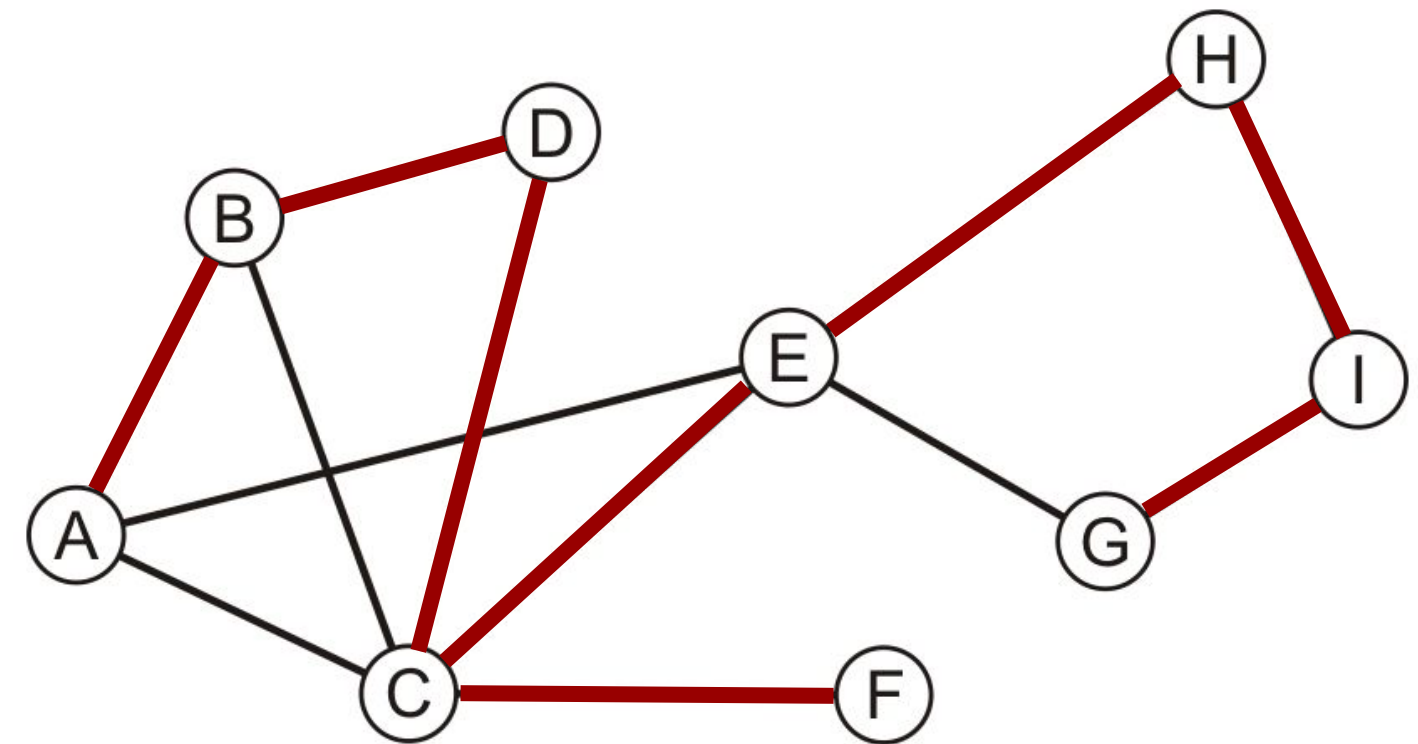


Búsqueda en profundidad (DFS)

Para explorar un grafo de esta manera vamos a necesitar una pila (stack)

DFS puede ser implementado de manera recursiva o iterativa

1. Elige cualquier vértice u , y agrégalo al stack marcándolo como visitado
 - a. Desde u , si hay algún vértice v que no haya sido visitado agrégalo al stack y continúa la búsqueda desde v
 - b. De otra forma, remueve el vértice u del stack y continúa desde el siguiente en la pila
2. Continúa el proceso hasta que no hayan más vértices que visitar



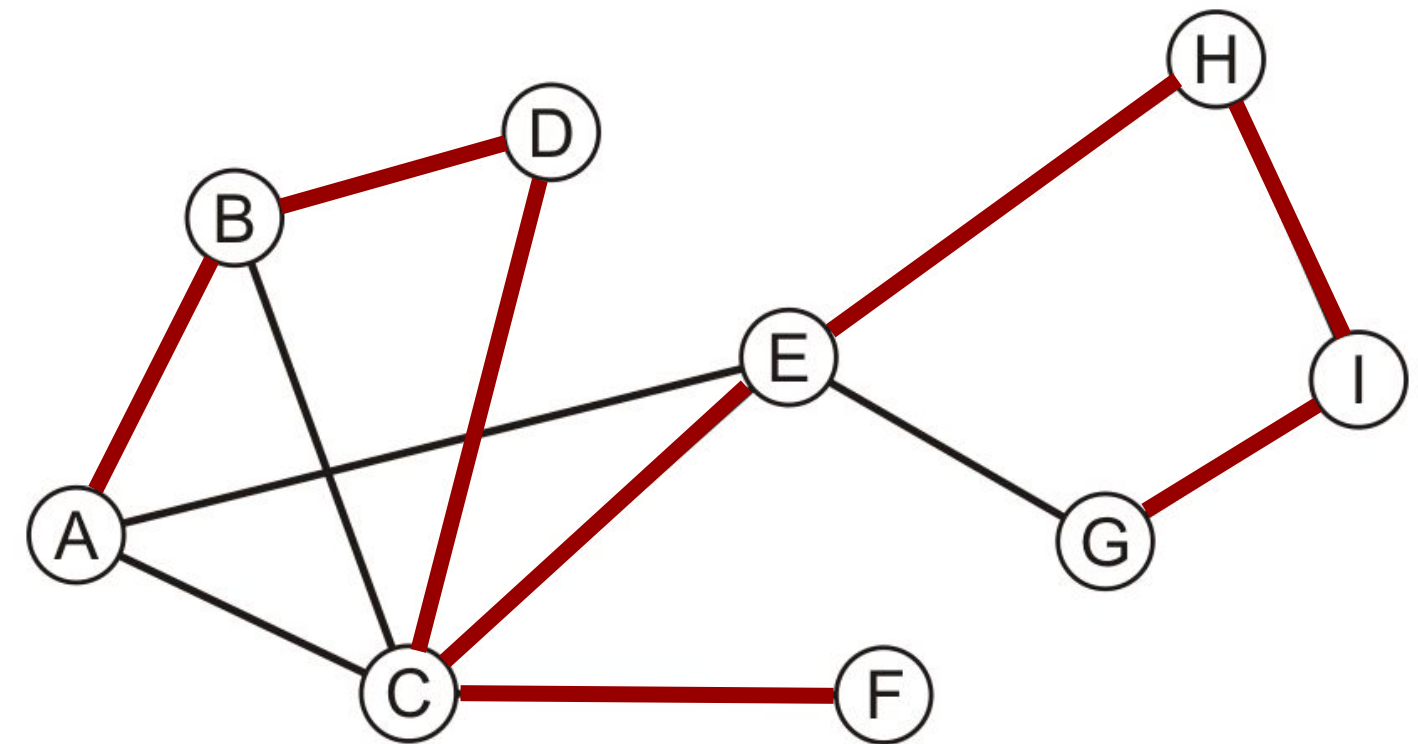
Búsqueda en profundidad (DFS)

```
function DFS(Graph G(V, E))
{
    for (i = 0; i < V.length; i++)
        visited[i] = false;

    for (i = 0; i < V.length; i++)
        if (!visited[i])
            DFS(G, i);
}

function DFS(Graph G(V, E), int i)
{
    visited[i] = true; //add to solution
    foreach(v[j] adjacent to v[i])
        if (!visited[j])
            DFS(G, j);
}
```

$O(|V|+|E|)$

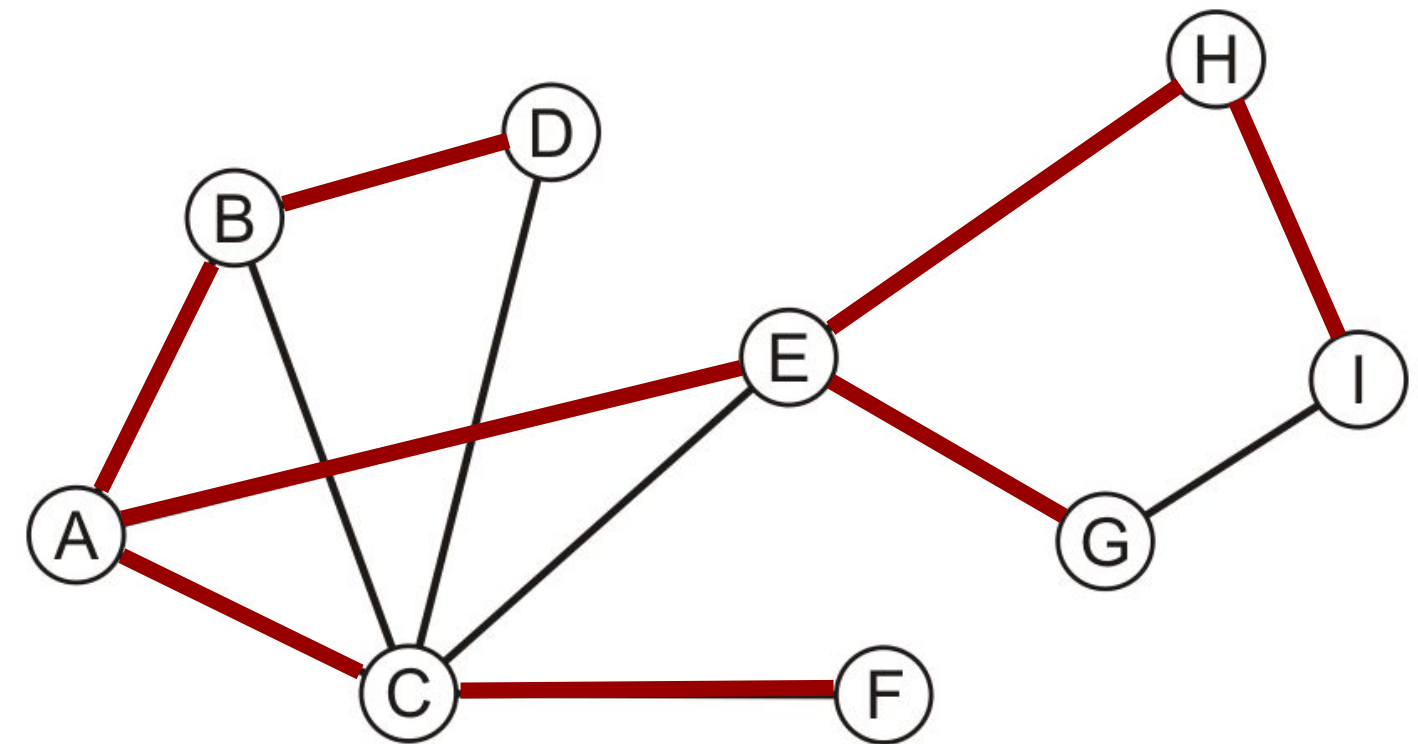


Búsqueda en amplitud (BFS)

Para explorar un grafo de esta manera vamos a necesitar una cola (queue)

El tamaño de la queue puede llegar a ser $O(|V|)$

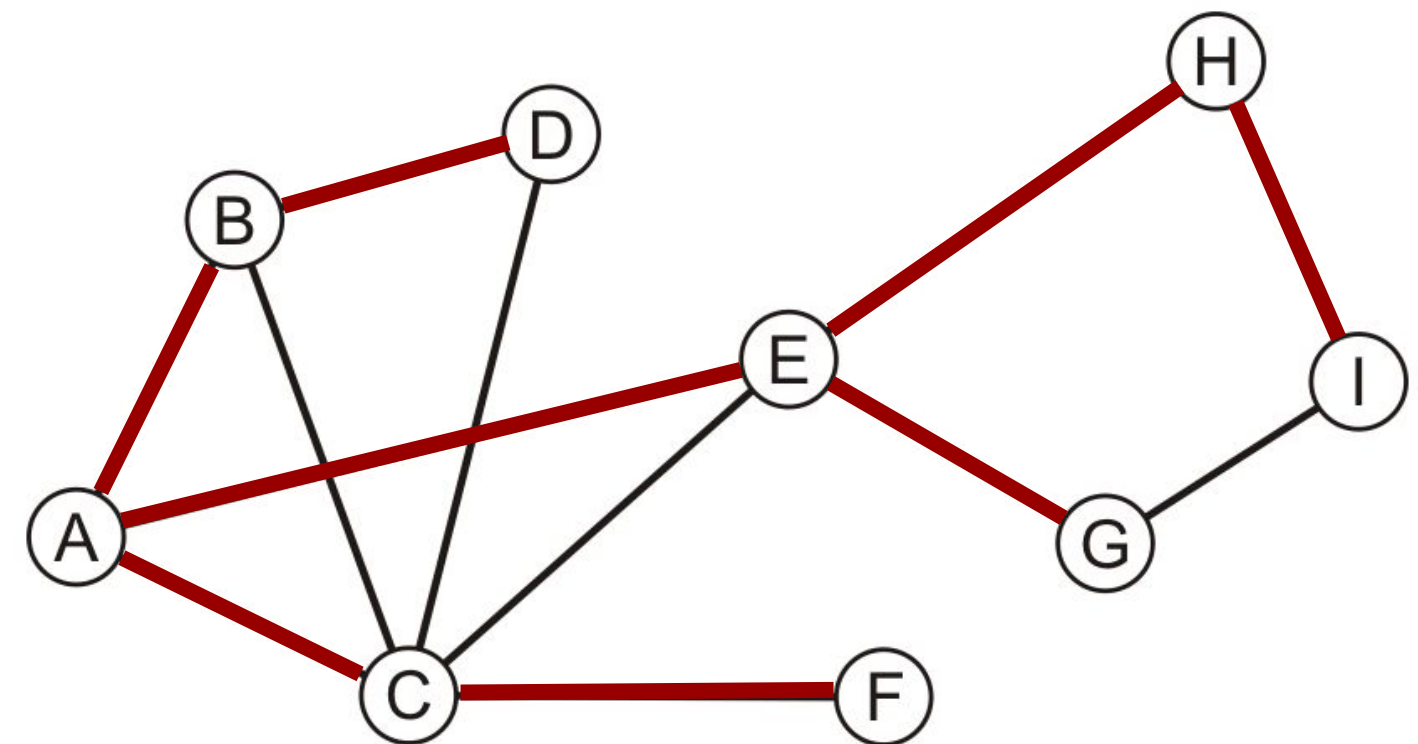
1. Elige cualquier vértice u , y agrégalo a la queue marcándolo como visitado
 - a. Pop el vértice v en la cima de la queue
 - b. Agrega todos los vértices adyacentes a v que no hayan sido visitados
 - c. Marca como visitados todos los vértices agregados a la queue
2. Continúa el proceso hasta que no hayan más vértices que visitar



Búsqueda en amplitud (BFS)

```
function BFS(Graph G(V, E))
{
    for (i = 0; i < V.length; i++)
        visited[i] = false;
    for (i = 0; i < V.length; i++)
        if (!visited[i])
            BFS(G, i);
}

function BFS(Grafo G(V, E), int i)
{
    Queue Q;
    visited[i] = true; // add to solution
    Q.add(i);
    while (!Q.empty()) {
        x = Q.extract();
        foreach(v[j] adjacent to v[x])
            if (!visited[j]) {
                visited[j] = true; // add to solution
                Q.add(j);
            }
    }
}
```



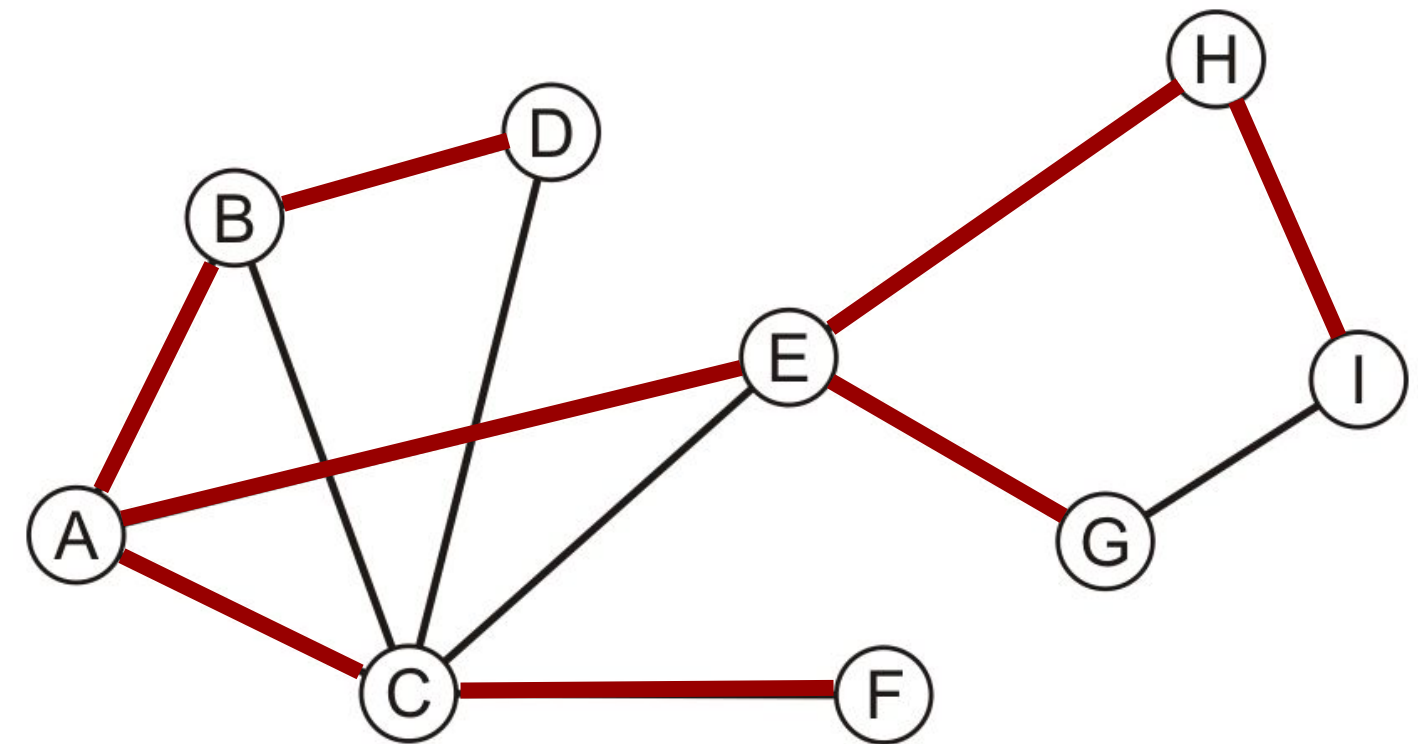
$O(|V|+|E|)$

Búsqueda en amplitud (BFS)

¿Cómo harían para obtener el camino de un vértice inicial al final?

Se tendría que guardar (quizás en un map) el padre del nodo actual para poder iterar hacia atrás. Por ejemplo:

$P(B) = A$, $P(E) = A$, $P(C) = A$, $P(D) = B$, $P(H) = E$, $P(G) = E$,
 $P(I) = H$, y $P(C) = F$



Comparación

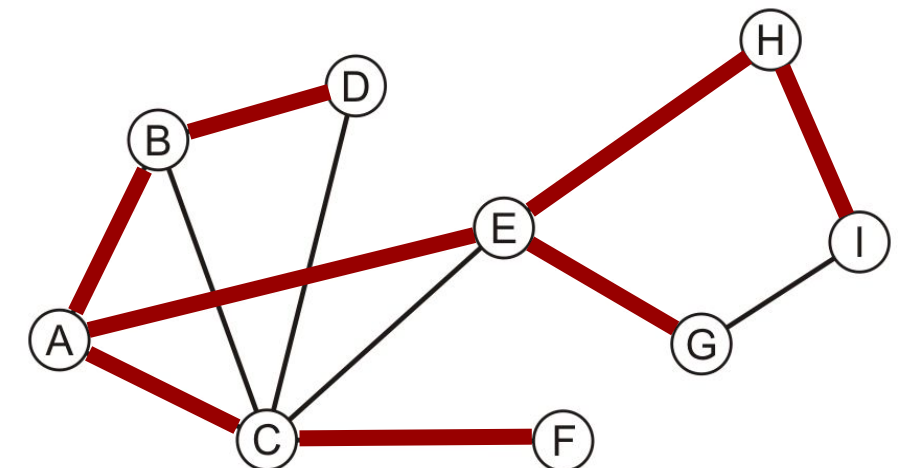
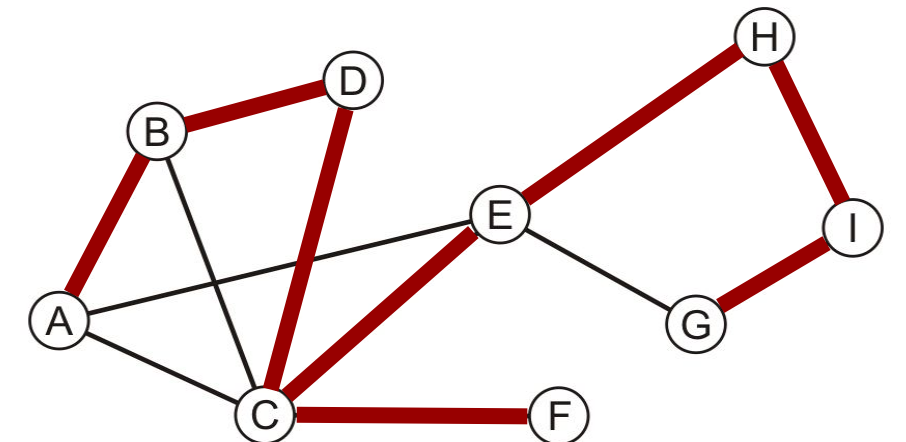
DFS es un algoritmo basado en vértices, mientras que BFS es basado en aristas. Utilizan estructuras de apoyo diferentes.

BFS puede encontrar el camino más corto en grafos con aristas del mismo peso, grafos bipartitos, etc

DFS se utiliza para obtener el ordenamiento topológico, saber si un grafo es fuertemente conexo

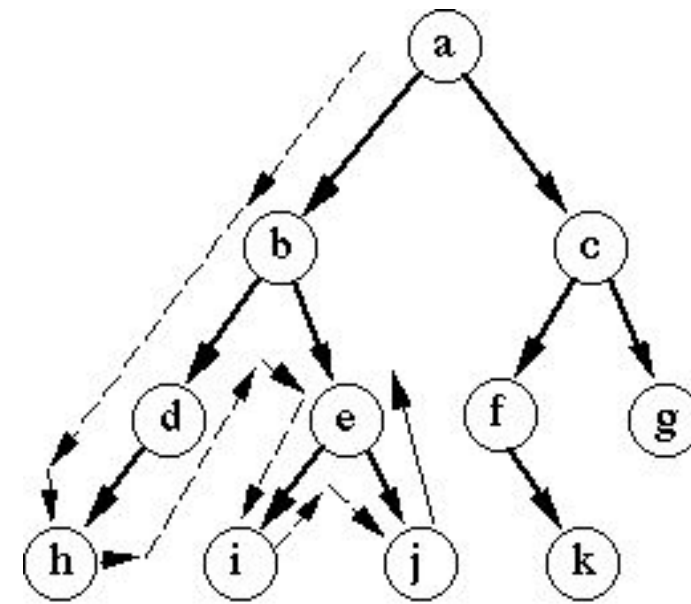
Cómo serán los árboles que generan?

DSF son árboles profundos y estrechos mientras que BFS produce árboles pequeños y amplios

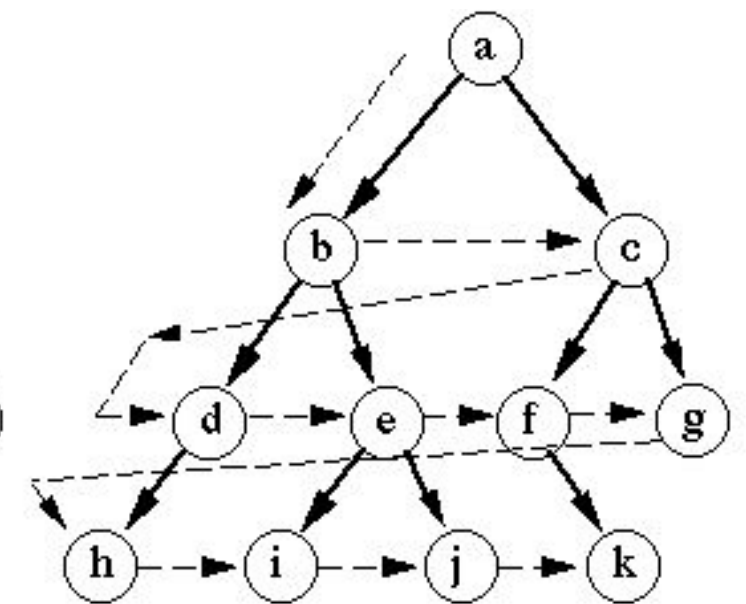


Aplicaciones

1. Determinar la conectividad de un grafo.
2. Encontrar el camino de un vértice a todos los demás
3. Probar si un grafo es bipartito
4. Análisis de una red y sus relaciones
5. Detección de ciclos
6. Ordenamiento topológico
7. Detección de árboles



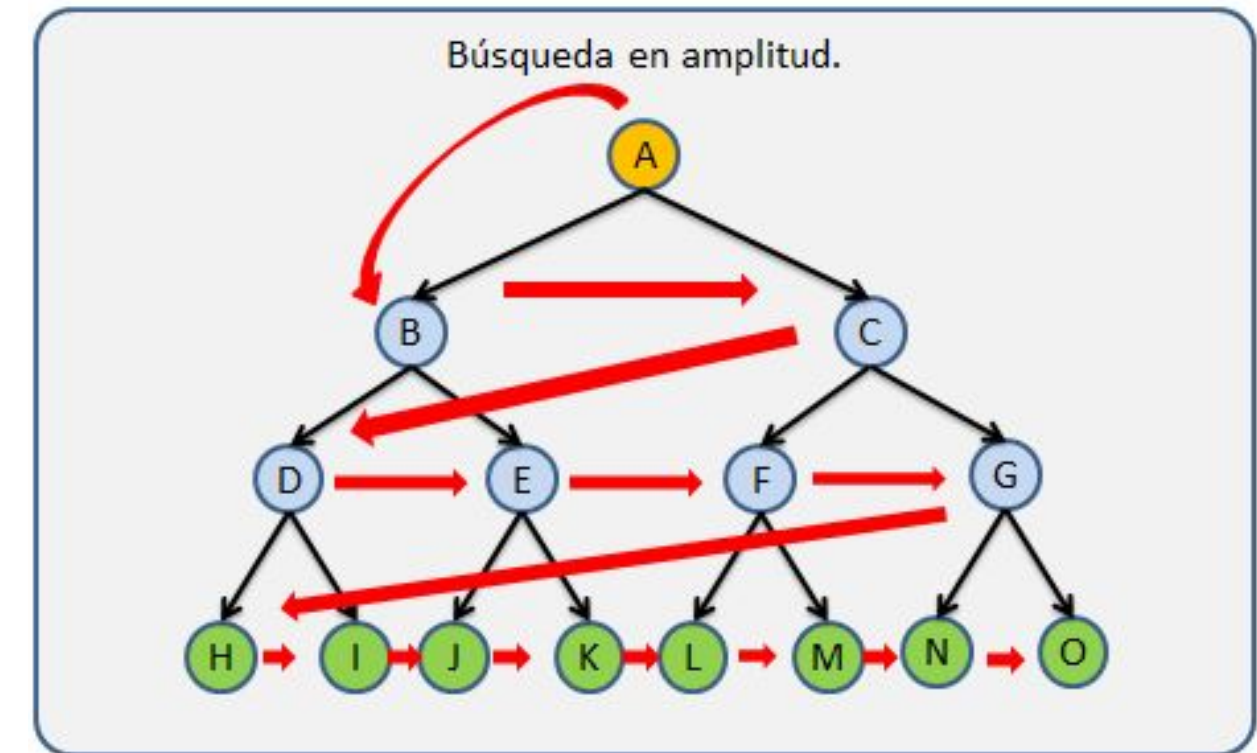
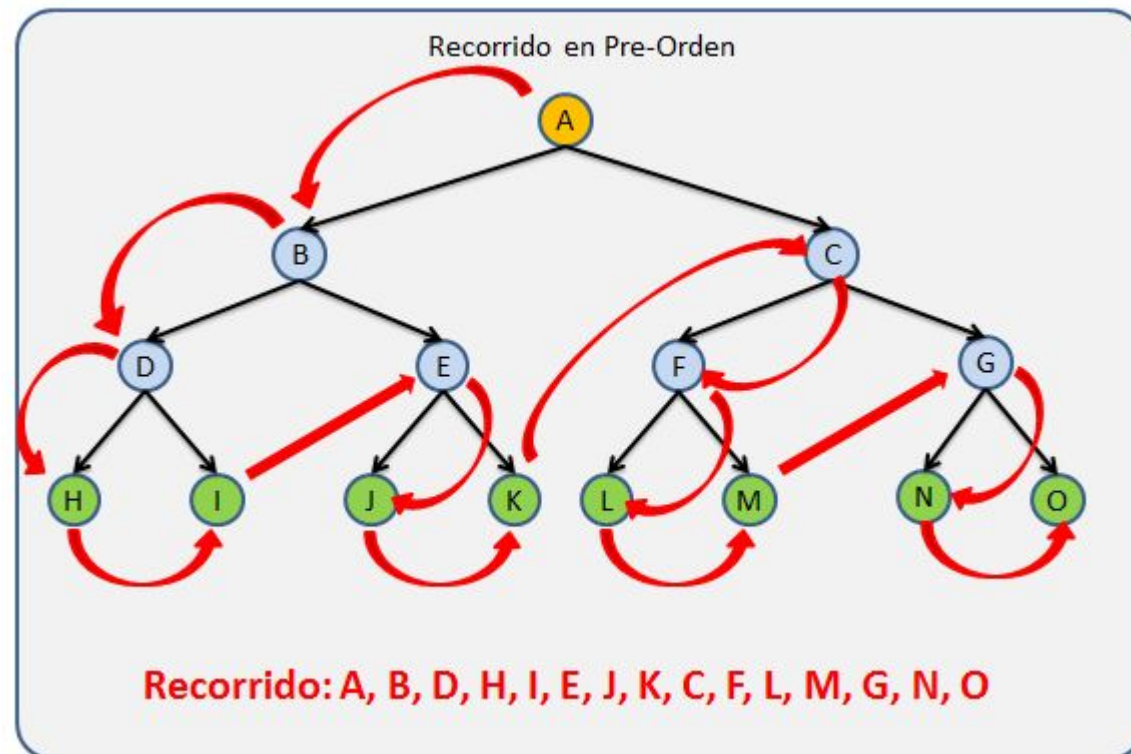
Depth-first search



Breadth-first search

Árboles

Entonces, cómo sería la búsqueda en profundidad y amplitud en un árbol?



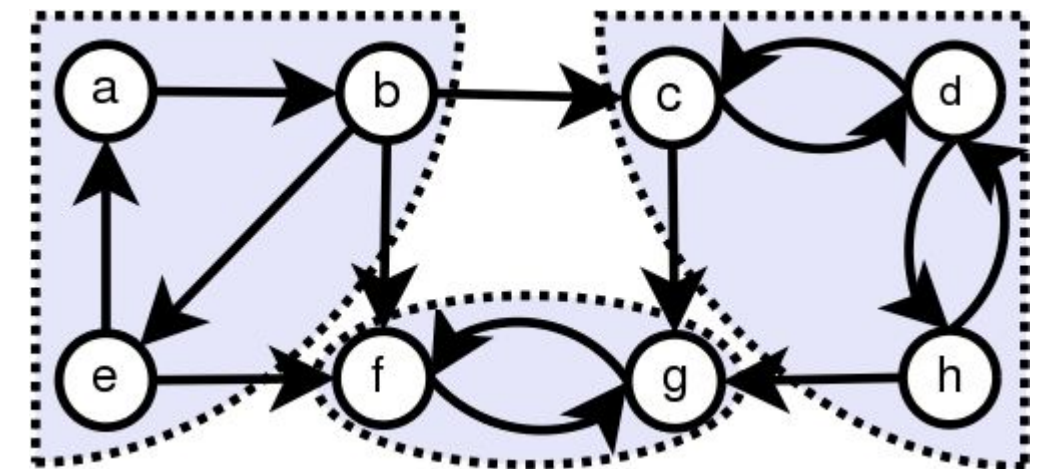
Componentes fuertemente conexos

- Desde cada nodo de la componente existe un camino hacia todos los otros nodos que usa aristas únicamente de la componente.
- No hay un nodo del grafo fuera de la componente desde el cual se pueda llegar a la componente y al que se pueda llegar desde ella (es “maximal”, si existiera estaría en la componente).

Componentes fuertemente conexos

El algoritmo sería:

1. Aplicar búsqueda en profundidad sobre el grafo G (se debe ir anotando los tiempos para cada caso de descubierta y cerrado del vértice)
2. Encontrar el grafo traspuesto G' (básicamente cambiar la dirección de las aristas de G)
3. Se ordena los vértices de mayor a menor tiempo de finalización
4. Aplicar búsqueda en profundidad sobre el grafo traspuesto G' . La búsqueda va sobre los vértices ordenados en el paso anterior
5. El resultado será un conjunto de componentes fuertemente conexos

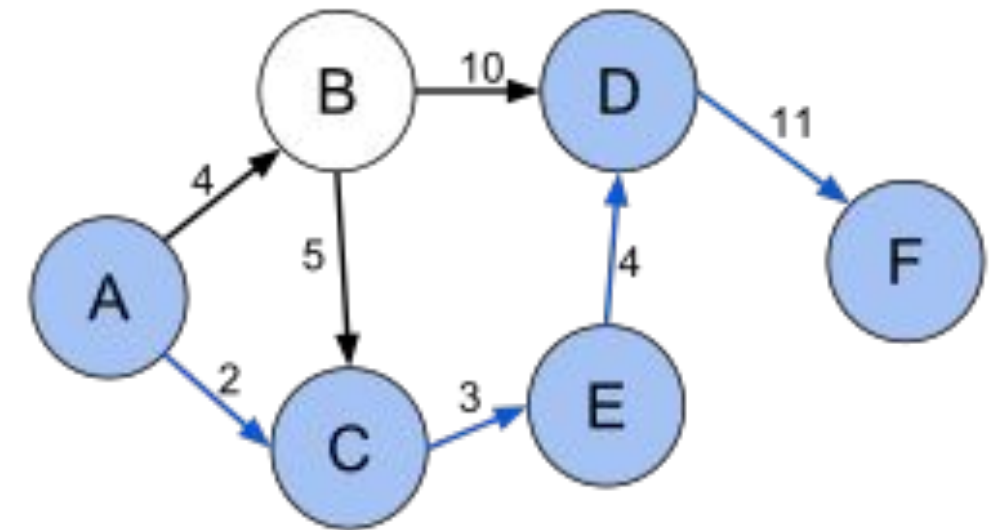


Camino más corto

Estos algoritmos buscan encontrar el camino entre dos vértices, tal que la suma de los pesos de las aristas sea el mínimo posible

BFS puede resolver este problema en un grafo no ponderado

El problema del camino más corto puede existir en grafos dirigidos o no dirigidos



Dijkstra

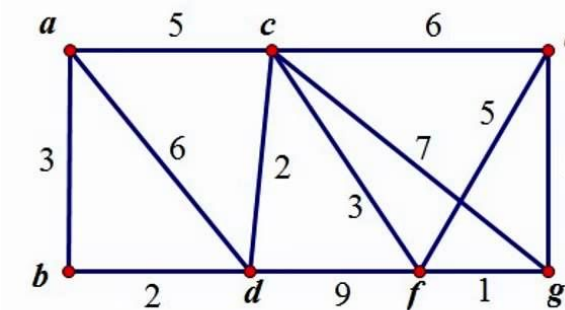
Es un algoritmo para obtener el camino mínimo en un grafo de un vértice a otro.

Este algoritmo solo funciona con aristas de valores positivos, ya que asume que agregar una arista sólo incrementará el peso total del camino

Básicamente construye un árbol de los caminos más cortos de un vértice fuente a todos los demás nodos

El algoritmo termina cuando se visitan todos los vértices

Se utiliza una tabla para mantener las distancias, las cuales serán inicialmente infinito excepto para el vértice fuente



<i>y</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>	<i>g</i>
<i>a</i>	0 _a	3 _a	5 _a	6 _a	∞ _a	∞ _a	∞ _a
<i>b</i>	0 _a	3 _a	5 _a	5 _b	∞ _a	∞ _a	∞ _a

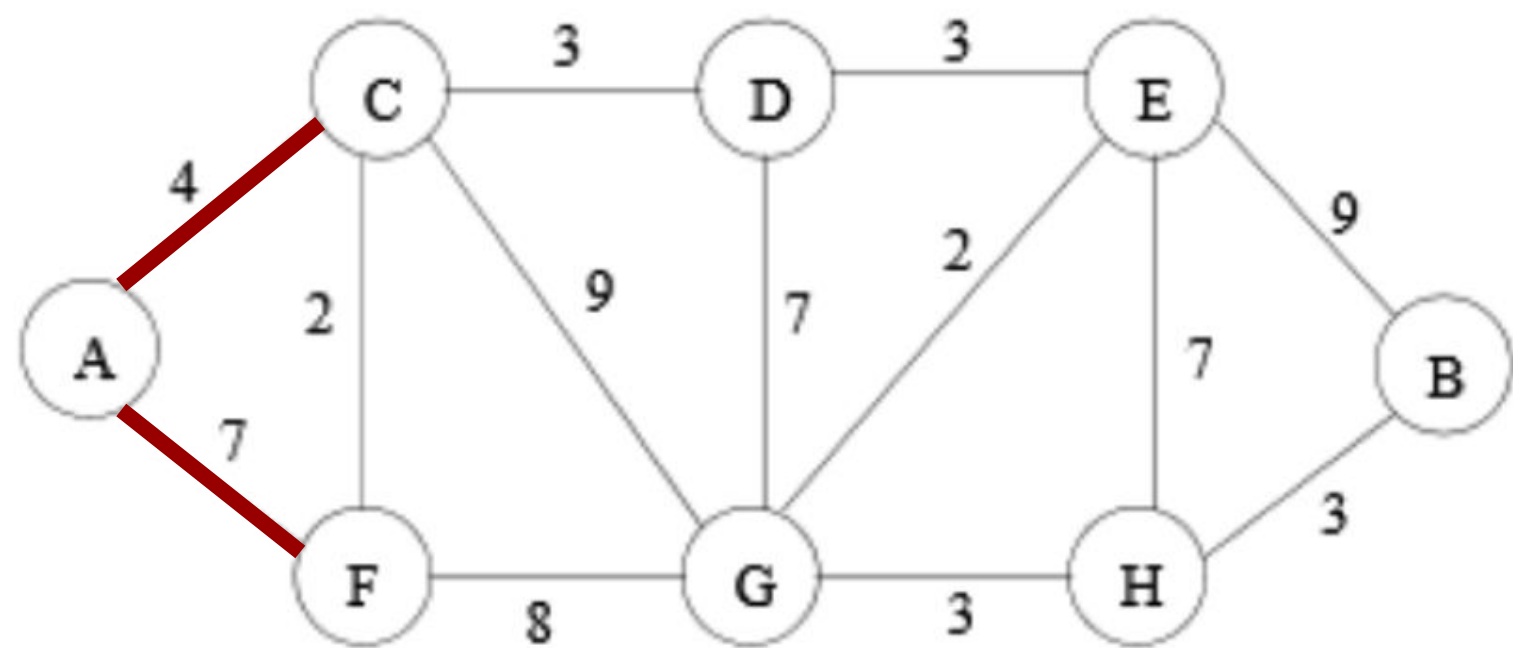
Dijkstra

Con un grafo conexo dirigido o no dirigido ponderado, se debe seleccionar un vértice inicial u . Se usará un arreglo de vértices visitados, y un arreglo para las distancias de un vértice x a uno y .

1. Todas las distancias entre todos los vértices empiezan con infinito, excepto el vértice u que iniciará con 0
2. Tomamos como *actual*, a nuestro vértice inicial u
3. Se recorren todos los vértices adyacentes a *actual*, excepto por los ya visitados y se calcula la distancia tentativa. Si la distancia tentativa es menor que la distancia almacenada, esta se actualiza.
4. Una vez se hayan completado todos los vértices adyacentes, *actual* se marca como visitado (un vértice visitado jamás será visto de nuevo).
5. Se toma como próximo *actual* el de menor distancia (se puede usar una cola de prioridad o un vector) y se regresa al paso 3, mientras existan vértices no visitados.

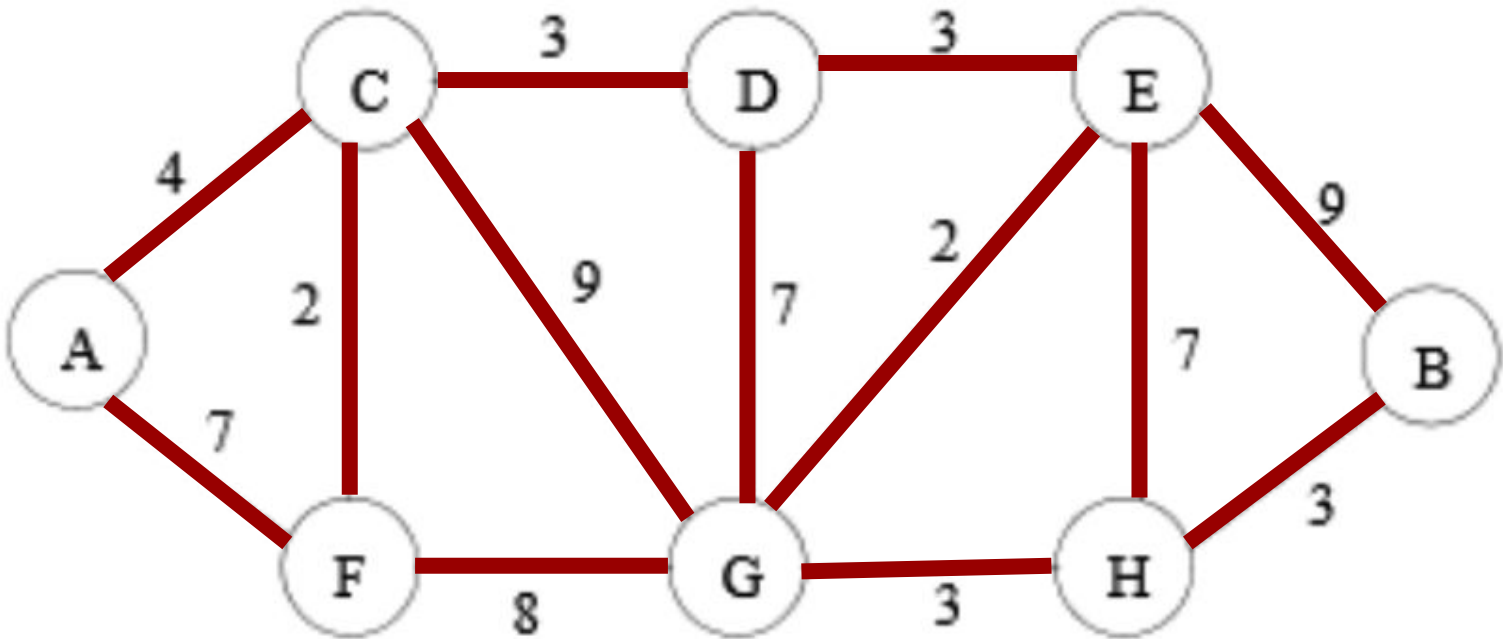
Una vez terminado al algoritmo, tendremos todas las distancias más cortas desde el vértice inicial u , a todos los demás vértices.

Dijkstra (A)



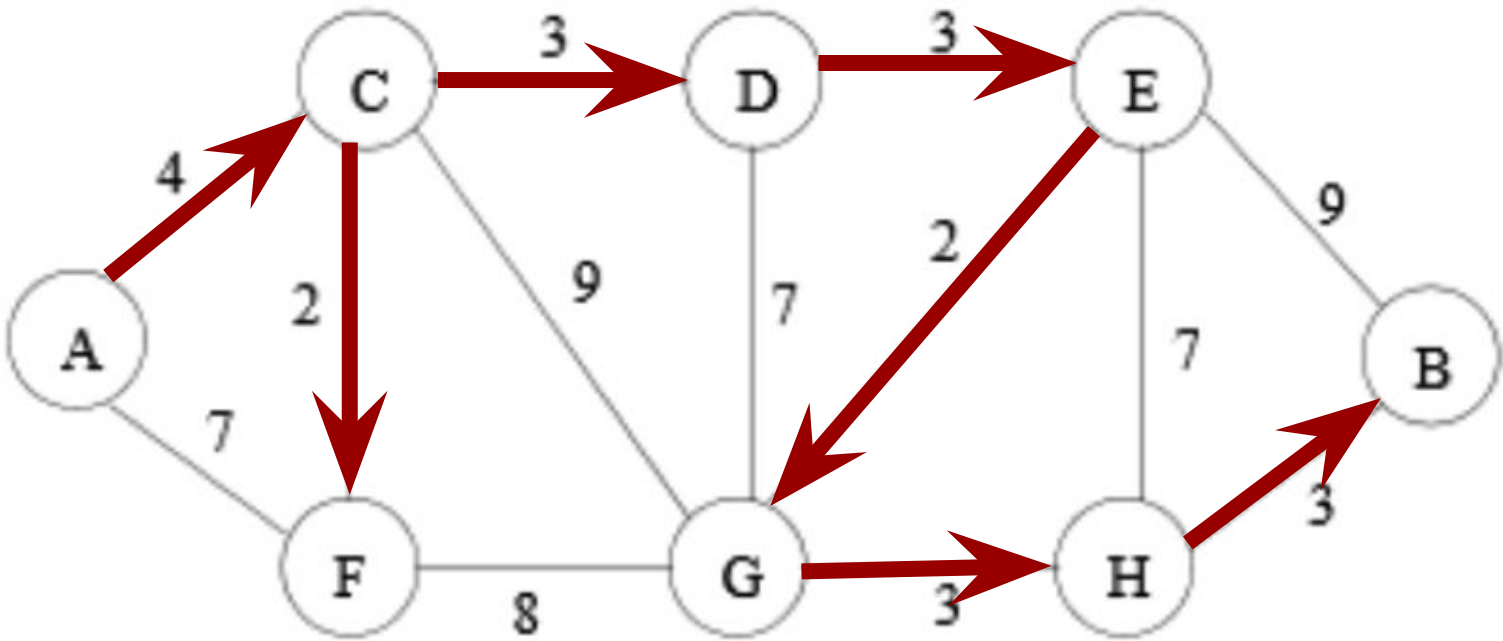
Actual	Distancia desde A							
	A	B	C	D	E	F	G	H
A	0	INF	INF	INF	INF	INF	INF	INF

Dijkstra (A)



Actual ↓	Distancia desde A							
	A	B	C	D	E	F	G	H
A	0	INF	4	INF	INF	7	INF	INF
C	0	INF	4	7	INF	6	13	INF
F	0	INF	4	7	INF	6	13	INF
D	0	INF	4	7	10	6	13	INF
E	0	19	4	7	10	6	12	17
G	0	19	4	7	10	6	12	15
H	0	18	4	7	10	6	12	15
B	0	18	4	7	10	6	12	15

Dijkstra (A)



Actual ↓	Distancia desde A							
	A	B	C	D	E	F	G	H
A	0	INF	4	INF	INF	7	INF	INF
C	0	INF	4	7	INF	6	13	INF
F	0	INF	4	7	INF	6	13	INF
D	0	INF	4	7	10	6	13	INF
E	0	19	4	7	10	6	12	17
G	0	19	4	7	10	6	12	15
H	0	18	4	7	10	6	12	15
B	0	18	4	7	10	6	12	15

Dijkstra (Algoritmo)

```
function Dijkstra(Graph G(V, E), u)
{
    for (i = 0; i < V.length; i++){
        distances[i] = INF;
        visited[i] = false;
    }
    distances[u] = 0;
    while(anyFalse(visited))
    {
        V[i] = minDistance(distances, visited);
        visited[i] = true;
        foreach(V[j] adjacent to V[i])
            if(distances[i] + G.weight(i, j) < distances[j])
                distances[j] = distances[i] + G.weight(i, j);
    }
}
```

Dijkstra (Algoritmo, guardando padre)

```
function Dijkstra(Graph G(V, E), u)
{
    for (i = 0; i < V.length; i++) {
        distances[i] = INF;
        visited[i] = false;
    }
    distances[u] = 0;
    while(anyFalse(visited))
    {
        V[i] = minDistance(distances, visited);
        visited[i] = true;
        foreach(V[j] adjacent to V[i])
            if(distances[i] + G.weight(i, j) < distances[j]){
                distances[j] = distances[i] + G.weight(i, j);
                parent[j] = i;
            }
    }
}
```

Ejercicios

Dibujen y apliquen los algoritmos a los siguientes grafos:

BFS(A)

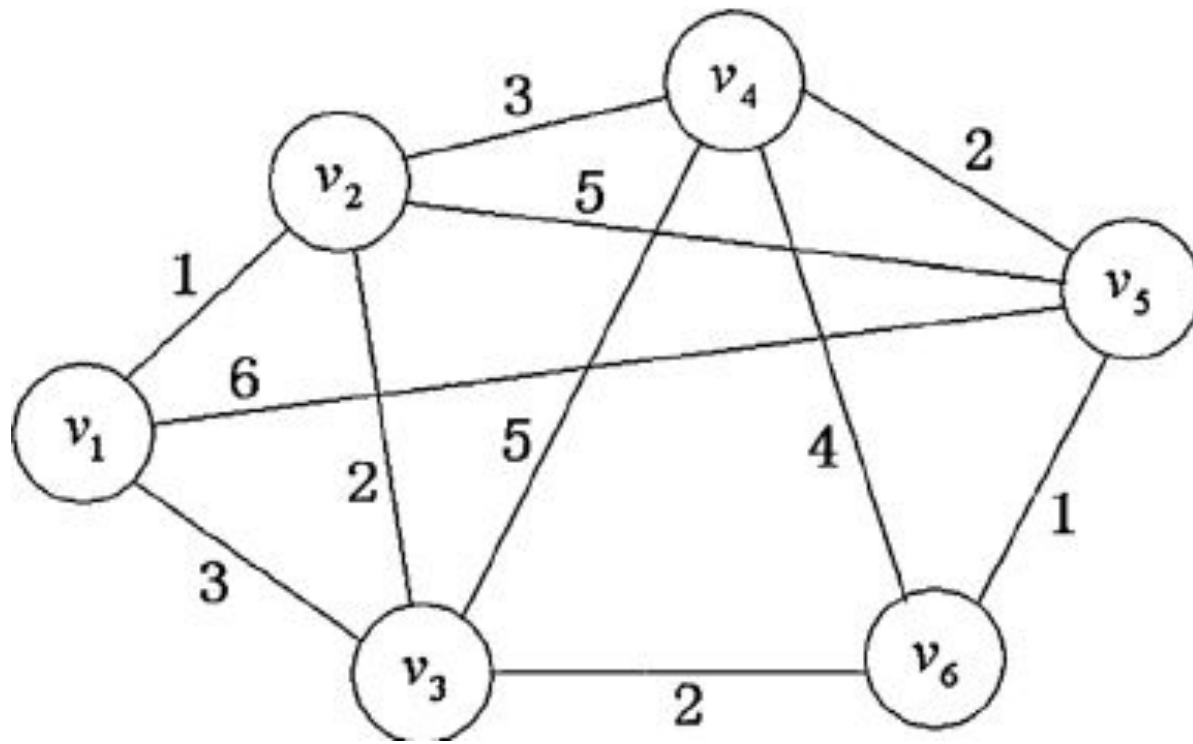
- ◆ $V = \{A, B, C, D, E, F\}$
- ◆ $E = \{(A, B), (A, E), (B, F), (C, A), (C, E), (E, B), (E, D), (F, E)\}$

DFS(F)

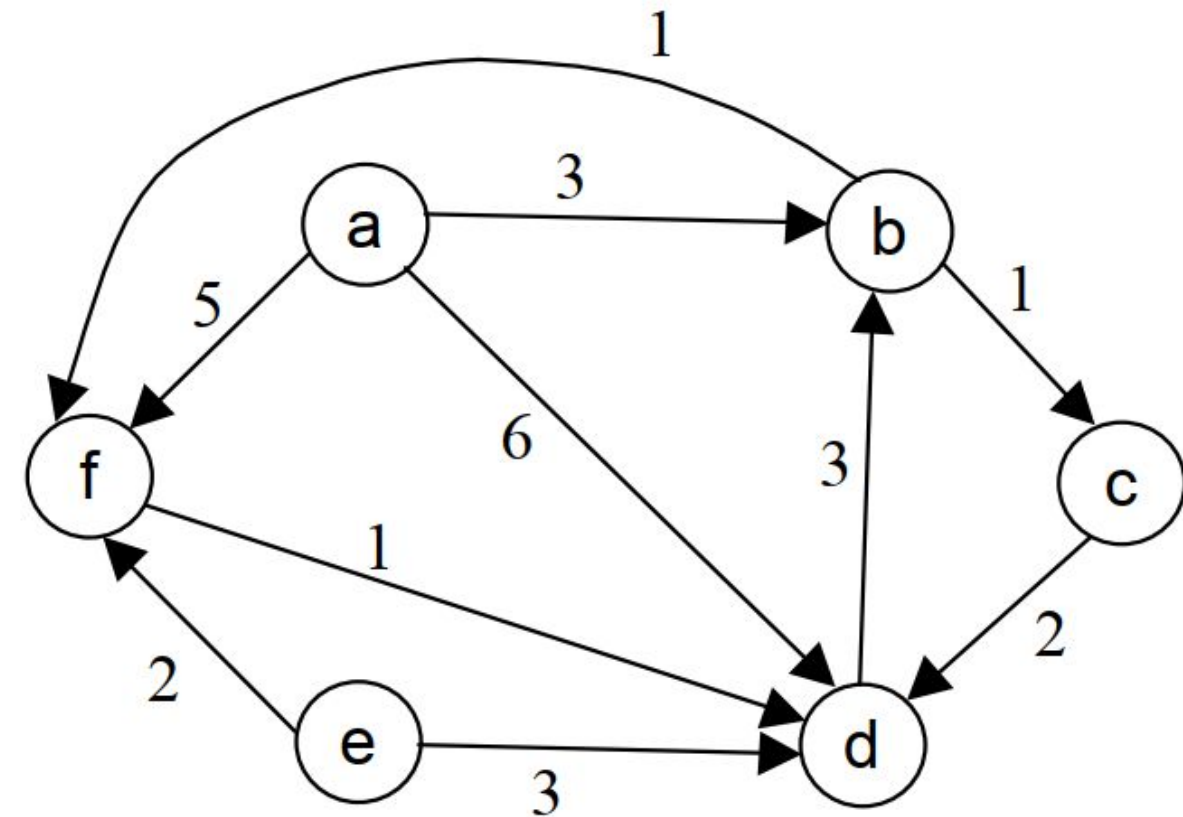
- ◆ $V = \{A, B, C, D, E, F\}$
- ◆ $E = \{\{A, B\}, \{A, C\}, \{A, F\}, \{B, D\}, \{B, C\}, \{B, F\}, \{C, E\}, \{C, D\}, \{D, F\}, \{E, F\}\}$

Ejercicios

Dijkstra (v1):



Dijkstra (a):



Solución

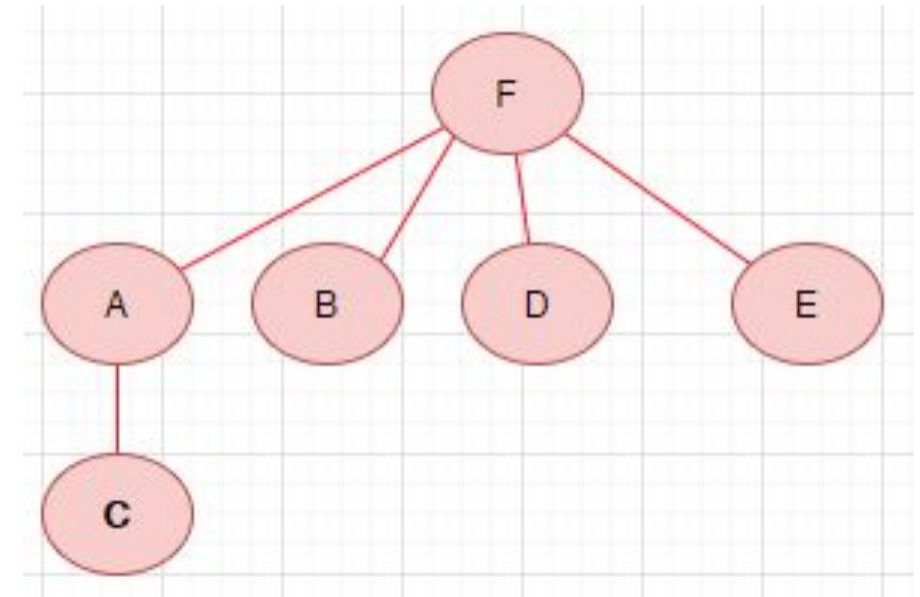
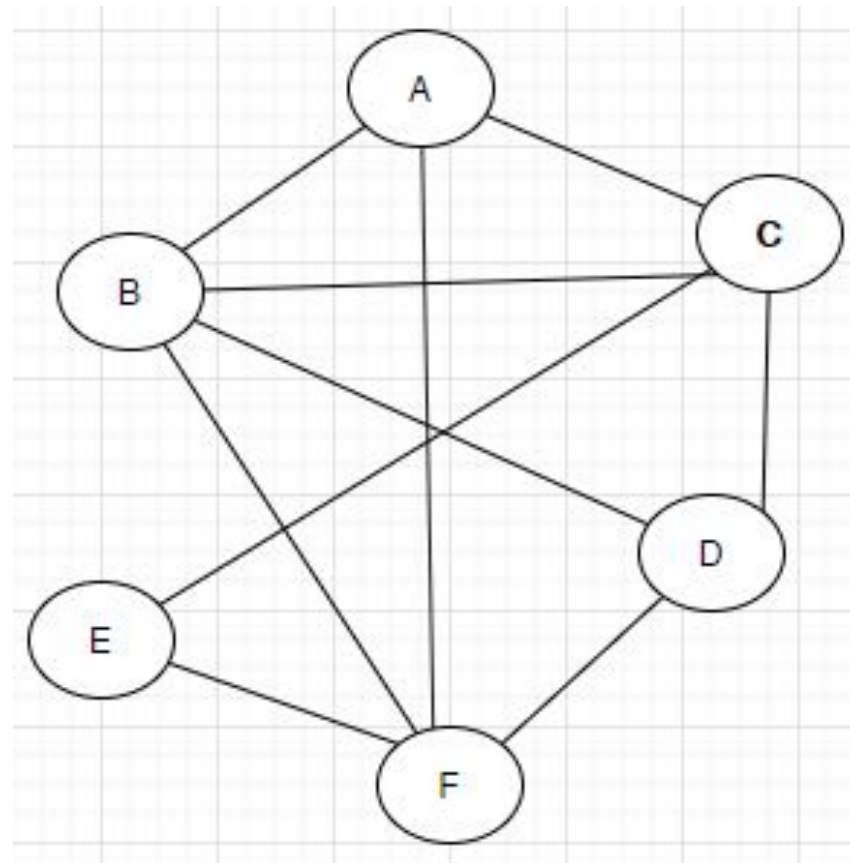


Solución

Dibujen y apliquen los algoritmos a los siguientes grafos:

BFS(F)

- ◆ $V = \{A, B, C, D, E, F\}$
- ◆ $E = \{\{A, B\}, \{A, C\}, \{A, F\}, \{B, D\}, \{B, C\}, \{B, F\}, \{C, E\}, \{C, D\}, \{D, F\}, \{E, F\}\}$

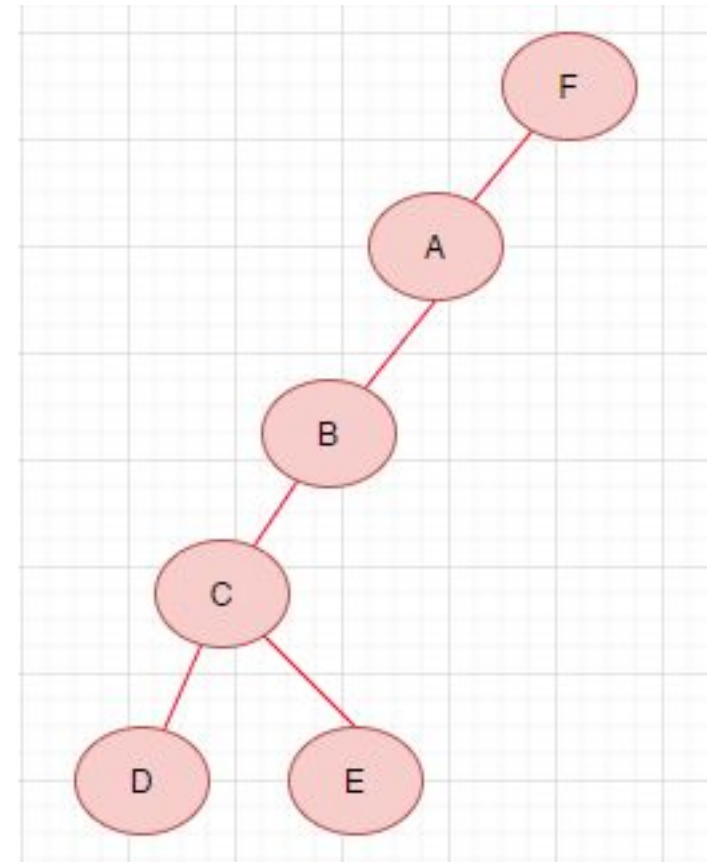
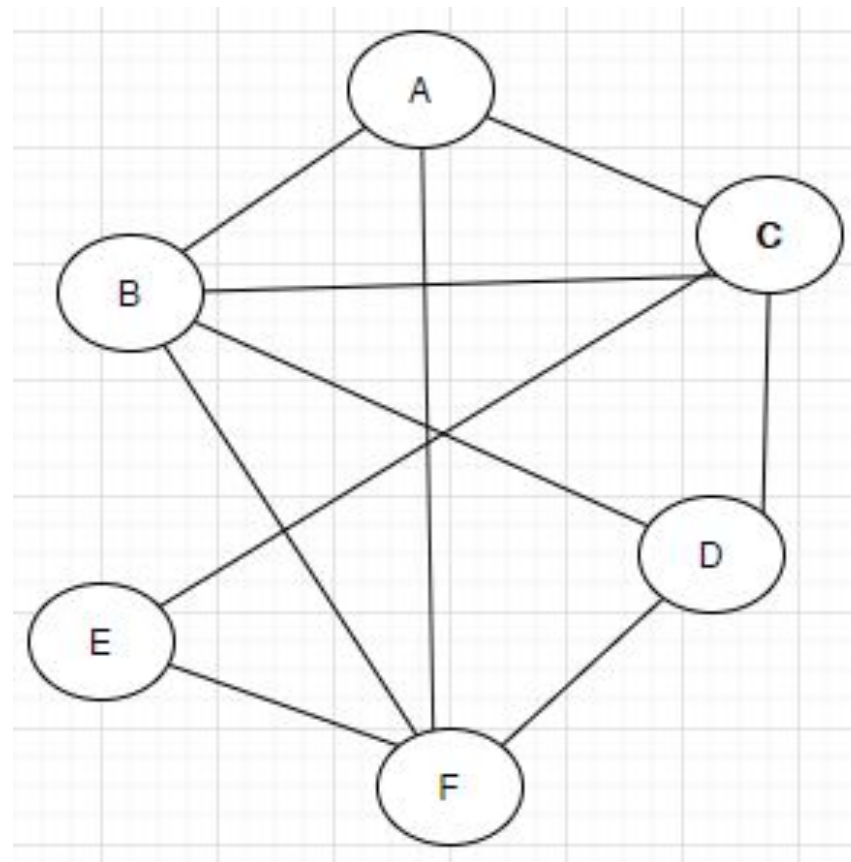


Solución

Dibujen y apliquen los algoritmos a los siguientes grafos:

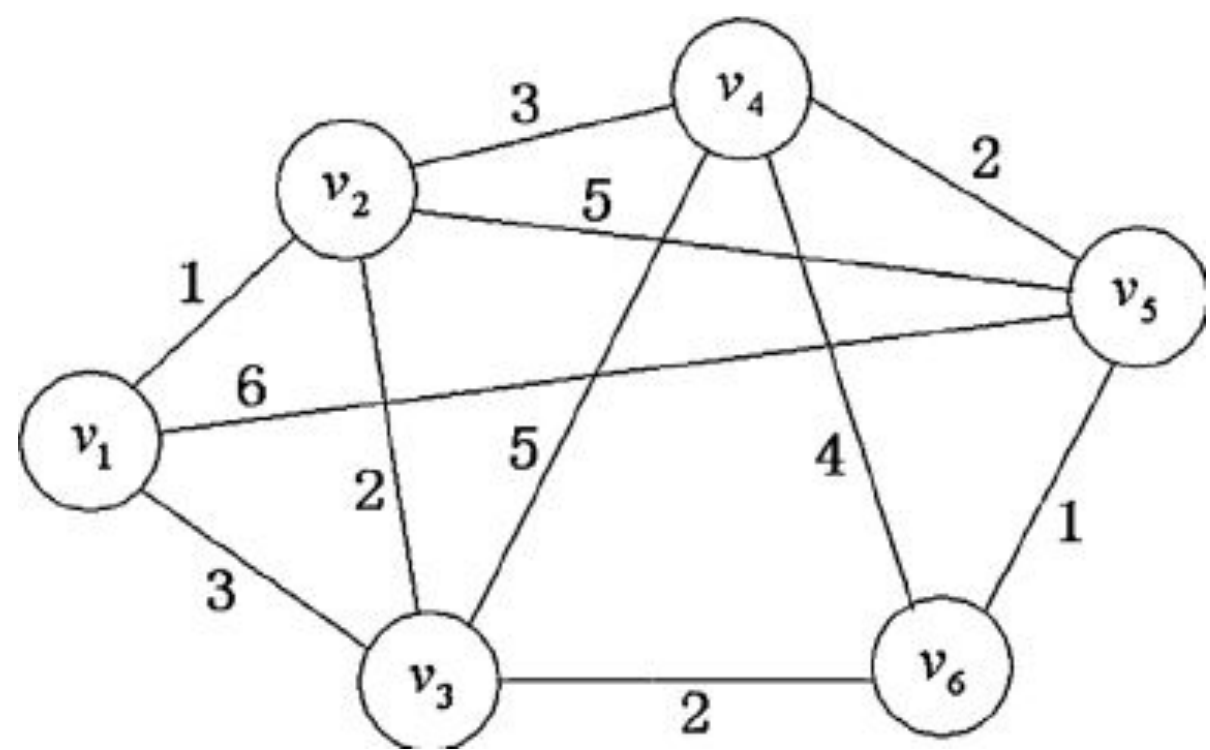
DFS(F)

- ◆ $V = \{A, B, C, D, E, F\}$
- ◆ $E = \{\{A, B\}, \{A, C\}, \{A, F\}, \{B, D\}, \{B, C\}, \{B, F\}, \{C, E\}, \{C, D\}, \{D, F\}, \{E, F\}\}$



Solución

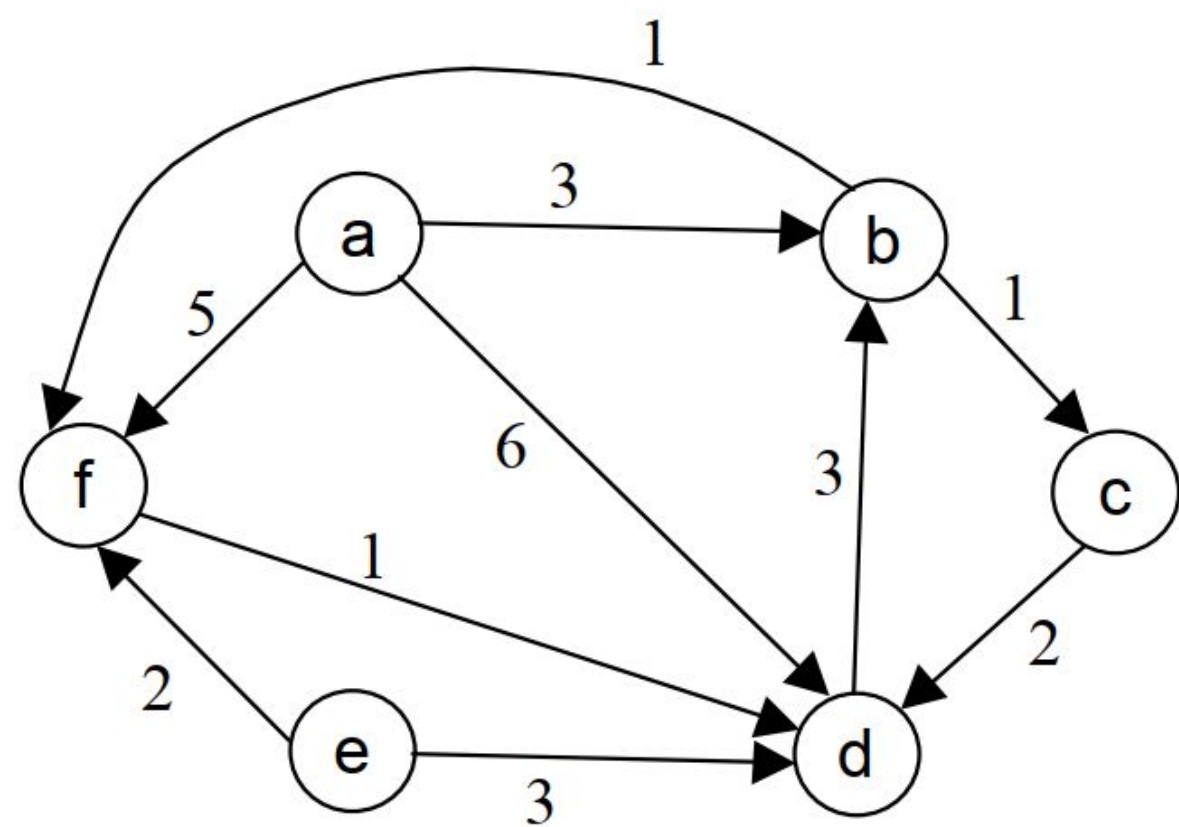
Dijkstra (v1):



Visitado	Distancia desde v1					
	v1	v2	v3	v4	v5	v6
v1	0	INF	INF	INF	INF	INF
v2	0	1	3	INF	6	INF
v3	0	1	3	4	6	INF
v4	0	1	3	4	6	5
v6	0	1	3	4	6	5

Solución

Dijkstra (a):



Visitado	Distancia desde a					
↓	A	B	C	D	E	F
A	0	INF	INF	INF	INF	INF
B	0	3	INF	6	INF	5
C	0	3	4	6	INF	4
F	0	3	4	6	INF	4
D	0	3	4	5	INF	4
E	0	3	4	5	INF	4